

terminal-server redesign - final version

February 28, 2024

Stephan Maka

stephan.maka@cyberus-technology.de

Lucas Baschinski

lucas.baschinski@cyberus-technology.de

Contents

1 Redesign proposal	2
1.1 Microservices Architecture	2
1.2 Architectural Sketch	2
1.3 Controlled peripherals	3
1.3.1 Step 1: 1:1 replacement ++	3
1.3.2 Step 2: feature add-ons from the HWaaS roadmap	3
1.3.3 Step 3: optional feature add-ons	3
1.4 Removed functionality	3
2 Time estimation	4
3 Priority List for "Step 2"	4
4 TODOs	5

1 Redesign proposal

This paper includes a multi-step proposal for the rewrite of the `TERMINAL-SERVER` in Rust. It is a follow-up to the `TERMINAL-SERVER` 2024 analysis paper. Please refer to it for the current state of implementation, potential alternatives with their up- and downsides and the analysis result.

Each service in the HWAAS ecosystem currently implements their own REST HTTP API. All peripherals are handled in specialized code on server and client sides. For preparing an abstraction, we will simplify non-user-facing services. The `TERMINAL-SERVER` OPENAPI schema will be removed, since the `net_ctrl` and `CONTEXT` API already generate their own descriptions. The internal interfaces will be handled in a generic way, so that the specific interface of a peripheral microservice will be reduced to configuration data.

Since the redesign is not supposed to enforce changes to the `CONTEXT` API, a way to connect the API calls to the new services is needed. This could be done by a reverse proxy for example.

1.1 Microservices Architecture

Currently, the `TERMINAL-SERVER` groups some peripherals into machines, HWAAS `CONTEXT` API dynamically groups machines into reservations. Boundaries between projects are already arbitrary: `CONTEXT` API duplicates reservation functionality, while `net_ctrl` is already another microservice that communicates with specialized hardware.

Therefore, we will decompose the functionality of the `TERMINAL-SERVER` into separately crashing microservices by splitting most individual peripheral functionality off into their own processes that each implement a proxy between the internal API and their respective peripheral. This allows for next to no impact on the `CONTEXT` API and `net_ctrl` interface.

Similar microservices in the same programming language may still share a repository because they will evolve together and share all the boilerplate code such as API data structures, API connectivity, config loading, and logging setup. Running microservices on a unified network API will allow replacing existing `TERMINAL-SERVER` functionality step by step, taking a few weeks per peripheral.

It is simpler to keep the reservation logic and state close to the user-facing `CONTEXT` API because that minimizes network-distributed locking and state synchronization. This is already the current state.

1.2 Architectural Sketch

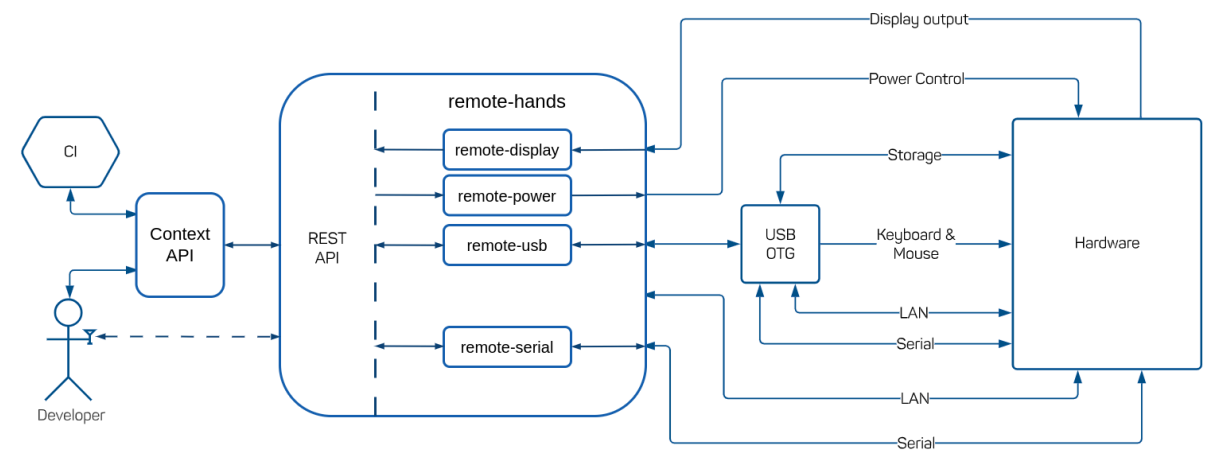


Figure 1: remote-hands system architecture design

1.3 Controlled peripherals

Tests must be able to communicate with various interfaces of the computer under test. The following peripherals will be implemented, taking into consideration the current usage of the `TERMINAL-SERVER` by the `HWAAS` and its users.

1.3.1 Step 1: 1:1 replacement ++

We decided to move HDMI capture up in priority. This will be the next work package, even though the 1:1 replacement is not fully done yet. Minor improvements like machine readable error messages can be done in parallel.

- **Serial** is implemented solely for TTY devices of the Linux host. A device read buffer can be obtained and also cleared. Read data is additionally broadcasted in real-time to WebSocket connections. Writing to the device can be accomplished by an API call and through WebSocket connections. Baudrate will be statically configurable and operation mode will be raw.
 - *Mostly done. Verification, testing and review will happen soon.*
- **HDMI** capture via additional hardware will be implemented.
 - Prototypes: Haskell (Cyberus internal), GO implementation from BLINDSPOT software
- **Power** provides methods prepare, start, and end. The adapter will be implemented for calling a custom shell command. All nix scripts inside the `HWAAS-DEPLOYMENT` repository will be reusable.
- **USB** is specific to the USB OTG Gadget infrastructure in Linux. Therefore, `TERMINAL-SERVER` must run on a Raspberry Pi 4. Images can be provided as USB mass storage devices (*virtual USB stick*). HID emulation is excluded. Must be dynamically configurable.
- **Auxiliary** devices are defined with a REST API and a shell command by which they can be enabled. Must be side effects free and resettable to a pre-defined state. Must comply with the documentation/specification.

1.3.2 Step 2: feature add-ons from the `HWaaS` roadmap

- **Power**: multiple power supplies, current measurements
- **USB**: Serial, mouse, keyboard support will be added. Additional support for ethernet, read-only mass-storage devices (for example: ISO read-only, image r+w) and WIFI is wanted. Mass-storage paths should change from a hash to a path.
 - HID Gadget Emulation in Rust: <https://docs.rs/hidg/latest/hidg/>
 - Prototypes: Haskell (Cyberus internal), GO implementation from BLINDSPOT software

1.3.3 Step 3: optional feature add-ons

- **Power**: Dedicated interfaces for IPMI, Redfish, and SNMP.

1.4 Removed functionality

- **Power**: AMT is not stable enough to be used in production. Since the `TERMINAL-SERVER` currently does not expose this functionality anyway, we can simply let it be.
- **Image server** and **reservation** functionality: The `CONTEXT` API implements these features as well, so the `TERMINAL-SERVER` doesn't need them anymore.
 - **reservation**: Reset to pre-defined state is currently handled via sessions. We could provide an endpoint that handles the reset or let the `CONTEXT` API do it (this could mean a deployment/downtime of the `CONTEXT` API for each new device that needs the reset handling). Will re-evaluate this when we got a bit farther into the rewrite and the state handling is implemented.

2 Time estimation

Assumption: 1 - 1.5 FTE for the TERMINAL-SERVER rewrite and implementation of additional features.

This concludes the following time estimation:

Topic	Time Estimation
Implementation “Step 1: 1:1 replacement”	Beginning of Q2
Real-world testing, hardening and deployment of Step 1 (complete removal of the old TERMINAL-SERVER)	Mid of Q2
Implementation “Step 2: feature add-ons from the HWAAS roadmap” ▪ Includes step-by-step deployment of added functionalities	End of Q3

3 Priority List for “Step 2”

1: high - 2: medium high - 3: medium - 4: medium low - 5: low

Category	Functionality	Priority
USB	Serial support	1
USB	Keyboard support	2
Power	multiple power supplies	2
Power	current measurements	3
USB	Mass-storage paths as paths instead of hashes	4
USB	Mouse support	4
USB	Ethernet support	5
USB	read-only mass-storage device support	5
?	WLAN support	5

4 TODOs

- ✓ Architecture sketch
- ✓ Time estimation for 3 FTE
- ✓ Priority list for “Step 2”
- ☐ API design for each remote-<function> service, since we provide an API to each device/peripheral
(will follow step-by-step for each feature before implementation)
- ✓ Get approval (Markus N., Philippe)